DOCUMENTACIÓN TÉCNICA Y MANUAL DE USUARIO

PLATAFORMA DE TENANTS «PRESENCIA CLOUD»

Aprovisionamiento automatizado del sistema de asistencia para múltiples instituciones

Proyecto final del módulo de DevOps e infraestructura como código

Elaborado por: Grupo GPT-6 AURA

Versión: 2.0

Fecha: 12 de noviembre de 2026

ÍNDICE

1. INTRODUCCIÓN 4

1.1. Antecedentes 4

1.2. Objetivo general 4

1.3. Objetivos específicos 4

1.4. Alcance 4

2. DESCRIPCIÓN GENERAL DE LA SOLUCIÓN 5

2.1. Concepto de tenant 5

2.2. Actores 6

2.3. Casos de uso 7

2.4. Glosario 8

3. ARQUITECTURA 9

3.1. Vista general 9

3.2. Componentes 10

3.3. Flujo de creación de un tenant 10

3.4. Función de cada herramienta 12

4. CATÁLOGO DE PLANES 13

5. INFRAESTRUCTURA COMO CÓDIGO 14

5.1. Imagen dorada con Packer y Ansible 14

5.2. Tenant con Terraform 14

5.3. Tenant con Pulumi 14

5.4. Configuración con cloud-init 14

5.5. Configuración posterior con Ansible 14

5.6. Despliegue de la plataforma 15

5.7. Almacenamiento y protección del estado 16

6. SEGURIDAD Y AISLAMIENTO 17

7. MONITOREO Y ALERTAS 18

8. MANUAL DE USUARIO **20**

8.1. Página de inicio y catálogo 20

8.2. Ingreso 21

8.3. Panel 22

8.4. Crear un tenant 22

8.5. Seguimiento del aprovisionamiento 23

8.6. Uso del tenant 24

8.7. Búsqueda y edición 26

8.8. Suspender, reanudar y eliminar 27

8.9. Reintentar una operación fallida 28

8.10. Alertas y eventos 29

9. PRUEBAS Y RESULTADOS **31**

10. CONCLUSIONES **32**

1. INTRODUCCIÓN

1.1. Antecedentes

Presencia es un sistema web de registro de asistencia desarrollado por el grupo para una institución. Cuando otras instituciones quieren usar el mismo sistema, instalarlo a mano para cada una es lento, propenso a errores y difícil de mantener. Este proyecto convierte Presencia en un servicio: una plataforma de administración crea, para cada institución, una copia independiente del sistema, llamada **tenant**, con su propia infraestructura en la nube.

1.2. Objetivo general

Construir una plataforma que aprovisione, configure, administre y monitoree tenants aislados de Presencia en Amazon Web Services, aplicando infraestructura como código de punta a punta.

1.3. Objetivos específicos

- Ofrecer un catálogo de planes con recursos definidos y elegir el plan al crear un tenant.
- Aprovisionar la red, los servidores y la base de datos con dos motores intercambiables: Terraform y Pulumi.
- Desplegar la aplicación desde una imagen preparada con Packer y Ansible, y configurarla con cloud-init y su conexión a la base de datos.
- Mostrar el progreso y los errores de forma comprensible, y permitir reintentar sin duplicar recursos.
- Suspender, reanudar y eliminar tenants manteniendo el control de costos.
- Monitorear la salud de cada tenant con Prometheus y Grafana, con alertas.
- Garantizar el aislamiento entre tenants y la protección de credenciales.

1.4. Alcance

Incluye la plataforma (consola web, API, procesamiento en segundo plano y monitoreo), los tres planes, los dos motores de aprovisionamiento, el ciclo de vida completo de los tenants, el escaneo de seguridad previo y el despliegue de la propia plataforma en AWS. Según lo acordado con el docente, quedan fuera la edición del catálogo desde la interfaz, la imposición de límites por plan, la migración entre planes y los respaldos o restauraciones manuales.

2. DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

2.1. Concepto de tenant

Un tenant es la instalación de Presencia de una institución. Cada uno tiene su propia red privada (VPC), su firewall, sus servidores, su base de datos, sus credenciales y su dirección web fija. Los usuarios de una institución no pueden acceder a los datos de otra. Al ingresar, el sistema se presenta con el nombre y la sigla de la institución.

2.2. Actores

Tabla 1. Actores del sistema

Actor	Descripción
Administrador de la plataforma	Crea y administra los tenants de todas las instituciones desde la consola.
Administrador del tenant	Persona de la institución que gestiona estudiantes, horarios y asistencia en su Presencia.
Estudiante	Registra su asistencia en el tenant de su institución.

2.3. Casos de uso

Tabla 2. Casos de uso del administrador de la plataforma

Código	Caso de uso	Descripción
CU-01	Consultar el catálogo de planes	Ver Básico, Estándar y Avanzado con sus recursos.
CU-02	Crear tenant	Registrar la institución, elegir plan y motor.
CU-03	Seguir el aprovisionamiento	Ver el log en vivo, el resultado y los errores.
CU-04	Consultar y buscar tenants	Filtrar por nombre, estado, plan y entorno; editar el nombre.
CU-05	Suspender tenant	Apagar los recursos y conservar los datos.
CU-06	Reanudar tenant	Volver a encender en la misma dirección.
CU-07	Eliminar tenant	Mostrar qué se borra y pedir confirmación.
CU-08	Consultar la salud en Grafana	Tablero por tenant y por entorno.
CU-09	Ver alertas y eventos	Alerta con el tenant afectado y la hora.
CU-10	Reintentar una operación fallida	Continuar sin duplicar recursos.
CU-14	Verificar el aislamiento	Comprobar que un tenant no accede a otro.

2.4. Glosario

Tabla 3. Términos utilizados

Término	Significado
Tenant	Instalación independiente de Presencia para una institución.
Plan	Tamaño del tenant: capacidad de cómputo, almacenamiento y base de datos.
Motor	Herramienta de infraestructura como código que crea los recursos: Terraform o Pulumi.
AMI	Imagen de servidor de AWS; la «imagen dorada» trae Presencia ya instalada.
VPC	Red privada virtual de AWS; cada tenant tiene la suya.
SSM Parameter Store	Almacén cifrado de AWS donde se guardan las contraseñas.
Operación	Tarea sobre un tenant: crear, suspender, reanudar o eliminar.

3. ARQUITECTURA

3.1. Vista general

La plataforma se ejecuta en una instancia EC2 propia y crea los tenants en la misma cuenta de AWS. Sus componentes corren en contenedores Docker orquestados con Docker Compose. El estado de Terraform y de Pulumi se guarda en S3 y los secretos en SSM Parameter Store.

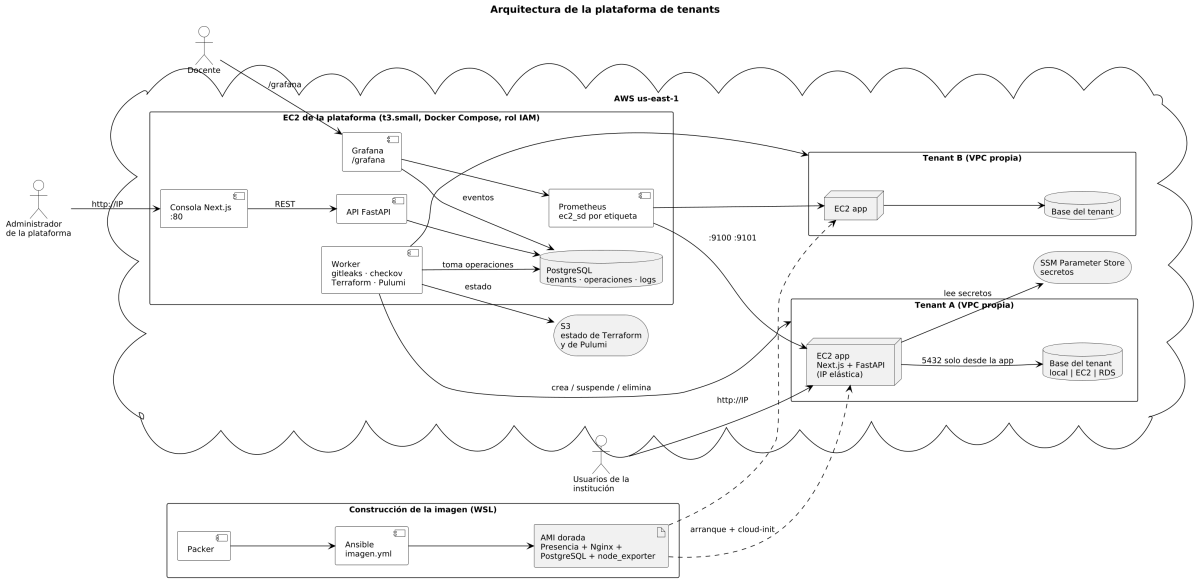


Figura 1. Arquitectura de la plataforma y de los tenants.

3.2. Componentes

Tabla 4. Componentes de la plataforma

Componente	Tecnología	Responsabilidad
Consola web	Next.js 16, React 19, shadcn/ui	Landing con planes, panel, gestión de tenants.
API	FastAPI, SQLAlchemy	Catálogo, tenants, operaciones, log, alertas y eventos.
Worker	Python, Terraform, Pulumi	Ejecuta las operaciones en segundo plano.
Base de la plataforma	PostgreSQL 18	Tenants, operaciones y su log.
Escáneres	gitleaks, checkov	Secretos y buenas prácticas antes de aprovisionar.
Monitoreo	Prometheus, Grafana	Métricas, tablero y alertas por tenant.
Imagen dorada	Packer, Ansible	AMI con Presencia, Nginx, PostgreSQL y node_exporter.
Configuración inicial	cloud-init	Datos del tenant y llamada al script de arranque.

3.3. Flujo de creación de un tenant

Al confirmar el formulario, la API registra el tenant y encola la operación. El worker la toma, busca la última imagen dorada, ejecuta los escáneres y luego el motor elegido. Cuando los servidores arrancan, cloud-init invoca el script que lee los secretos, prepara la base de datos, aplica las migraciones, crea el administrador y enciende la aplicación. El worker espera a que la aplicación responda y marca el tenant como activo.

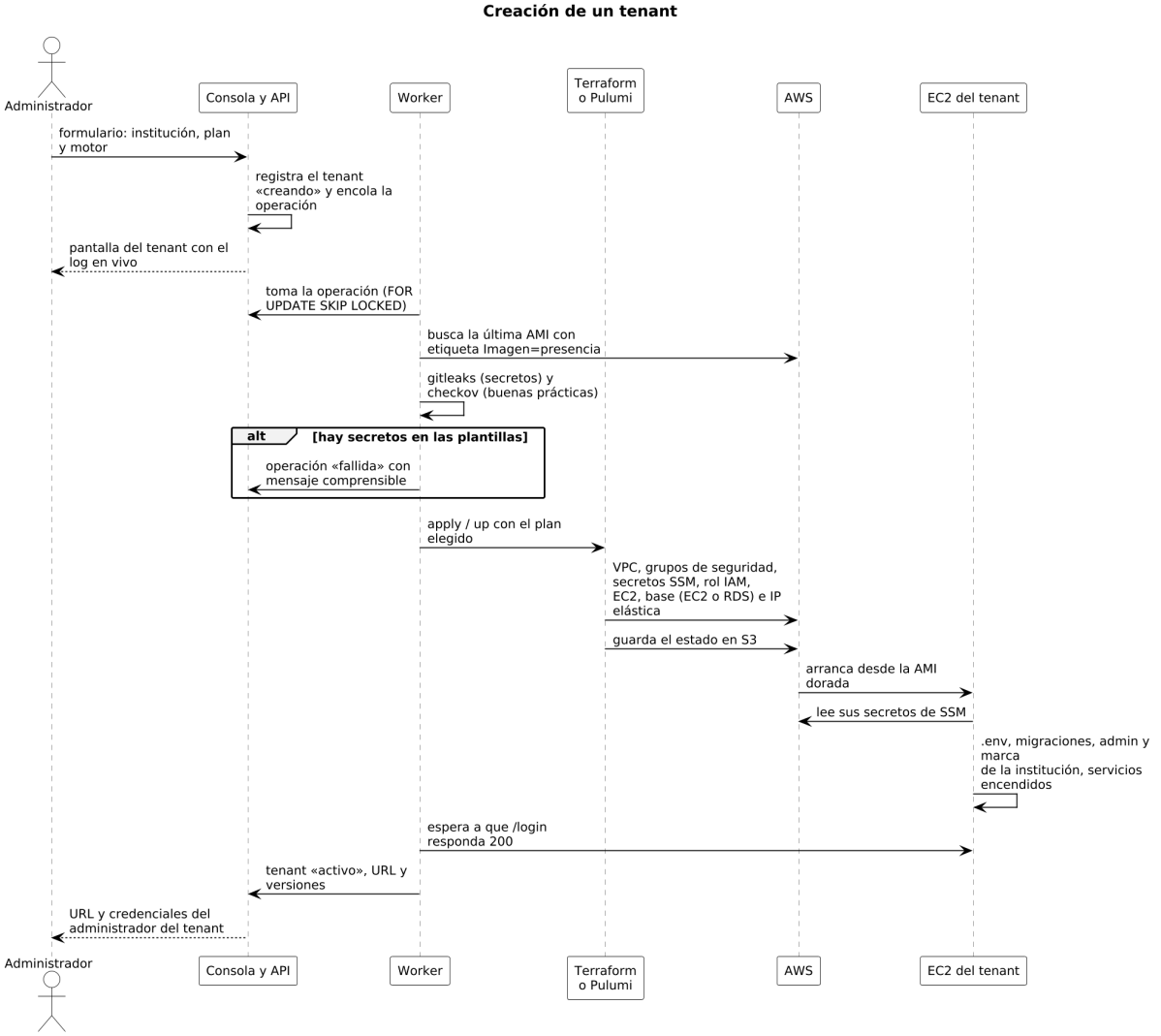


Figura 2. Secuencia de creación de un tenant.

3.4. Función de cada herramienta

Tabla 5. Herramientas y su función en la arquitectura

Herramienta	Función en el proyecto
Terraform	Motor principal: crea la red, la seguridad, los secretos, los servidores, la base y la IP de cada tenant, y la propia plataforma.
Pulumi	Segundo motor, en Python: crea exactamente los mismos recursos que Terraform para comparar ambos enfoques.
Packer	Construye la imagen dorada (AMI) de la que arrancan todos los servidores de los tenants.
Ansible	Dentro de Packer instala y compila Presencia en la imagen; después de crear cada tenant configura sus servidores: firewall, SSH, usuario de operación, actualizaciones y verificación de servicios.
cloud-init	Configuración inicial al primer arranque: deja los datos del tenant y ejecuta el script que lee los secretos, prepara la base y enciende la aplicación.
Grafana y Prometheus	Recolección de métricas, tablero de salud por tenant y por entorno, y alertas operativas.
gitleaks y checkov	Escaneo de secretos y de buenas prácticas antes de cada aprovisionamiento y en cada push del repositorio.
Docker Compose	Ejecuta los servicios de la plataforma en su servidor.

4. CATÁLOGO DE PLANES

El catálogo se define en un único archivo (plataforma/planes.yaml) que leen la consola, la API, Terraform y Pulumi, de modo que lo que se ofrece es exactamente lo que se aprovisiona. Los costos son aproximados en la región us-east-1 con los servidores encendidos.

Tabla 6. Catálogo de planes

Plan	Servidor de la aplicación	Base de datos	Alumnos	Costo por día
Básico	t3.micro: 2 vCPU, 1 GiB, disco 10 GB	PostgreSQL en el mismo servidor, dentro de sus 10 GB	Hasta 100	US$ 0,40
Estándar	t3.small: 2 vCPU, 2 GiB, disco 10 GB	Servidor EC2 t3.micro propio (2 vCPU, 1 GiB), 20 GB, accesible solo desde la aplicación	Hasta 1.000	US$ 1,05
Avanzado	t3.small: 2 vCPU, 2 GiB, disco 10 GB	Amazon RDS db.t3.micro, PostgreSQL 17, 20 GB, respaldo automático de 1 día	Hasta 10.000	US$ 1,15

Tabla 7. Elementos incluidos en todos los planes

Elemento	Detalle
Red	VPC propia, subred, puerta a internet y tabla de rutas.
Firewall	Web pública en el puerto 80; SSH y métricas solo para la IP de la plataforma.
Dirección fija	IP elástica: la URL no cambia al suspender y reanudar.
Credenciales	Contraseña de base y clave JWT únicas, cifradas en SSM Parameter Store.
Monitoreo	node_exporter y métricas de la API, visibles en Grafana.
Acceso de soporte	Session Manager de AWS y el usuario «operador» creado por Ansible.

Un tenant suspendido solo paga sus discos, la IP elástica y, en el plan Avanzado, el almacenamiento de la base de datos. La plataforma, en una instancia t3.small, cuesta aproximadamente US$ 0,70 por día.

5. INFRAESTRUCTURA COMO CÓDIGO

5.1. Imagen dorada con Packer y Ansible

Packer (`infraestructure/packer`) levanta una instancia temporal de Ubuntu 24.04 y la configura con el playbook de Ansible `imagen.yml`: instala PostgreSQL, Nginx, node_exporter, la CLI de AWS, uv y Node.js, copia el código, instala dependencias y compila el frontend. El resultado es una AMI cifrada y etiquetada con la versión del código, de la cual arrancan todos los tenants en pocos minutos.

5.2. Tenant con Terraform

El módulo raíz `infraestructure/tenant/terraform` reutiliza los módulos de red y de EC2 del proyecto. Según el plan crea la base local, un servidor de base o una instancia RDS, y guarda el estado en S3 con bloqueo nativo, una clave por tenant.

5.3. Tenant con Pulumi

El programa en Python `infraestructure/tenant/pulumi` crea exactamente los mismos recursos, nombres y etiquetas. Cada institución es un stack y el estado se guarda también en S3.

5.4. Configuración con cloud-init

La misma plantilla de cloud-init sirve a ambos motores. Solo escribe datos no secretos y ejecuta el script de arranque, que lee las contraseñas desde SSM con el rol IAM del servidor; ninguna credencial viaja en los datos de usuario.

5.5. Configuración posterior con Ansible

Cuando la aplicación del tenant responde, el worker ejecuta el playbook `tenant.yml` sobre cada servidor del tenant. La plataforma tiene su propio par de claves SSH: la clave pública se instala al crear los servidores y el puerto 22 solo se abre para la IP de la plataforma. El resultado de Ansible queda en el log de la operación.

Tabla 8. Tareas de la configuración posterior

Tarea	Detalle
Identidad	Nombre presencia-<tenant>-<rol> y zona horaria America/La_Paz.
Usuario de operación	«operador», sin contraseña, con sudo limitado a revisar y reiniciar los servicios.
SSH	Sin contraseñas ni acceso de root, máximo 3 intentos y solo usuarios autorizados.
Actualizaciones	Actualizaciones de seguridad automáticas.
Firewall (ufw)	SSH y métricas solo desde la plataforma; web pública; PostgreSQL solo dentro de la VPC; el resto denegado.
Verificación	Backend, frontend y Nginx corriendo; PostgreSQL aceptando conexiones.

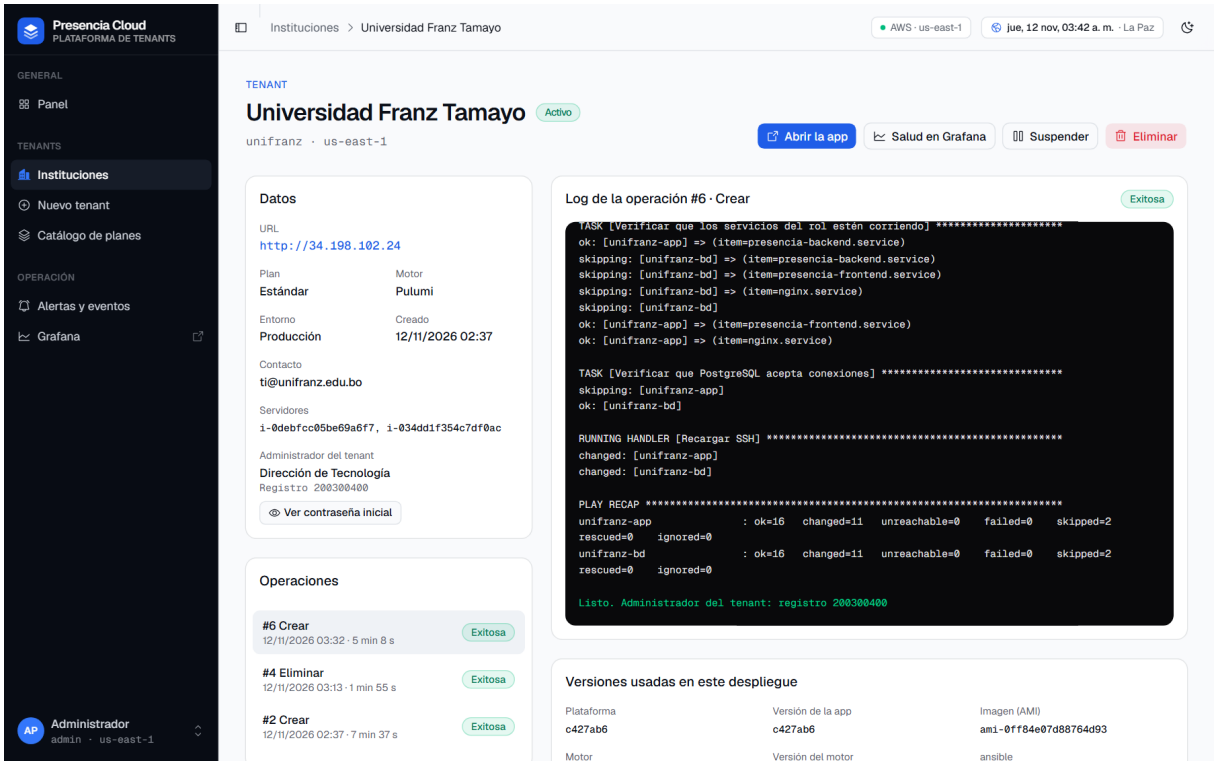


Figura 3. Resultado de Ansible en el log de creación de un tenant Estándar.

5.6. Despliegue de la plataforma

El ambiente `infraestructure/environments/plataforma` crea con Terraform la instancia de la plataforma, su firewall (solo puerto 80), su IP elástica, sus secretos y un rol IAM con los permisos mínimos para aprovisionar tenants. Su cloud-init instala Docker, clona el repositorio y levanta todos los servicios.

5.7. Almacenamiento y protección del estado

El estado de los motores registra qué recursos existen en AWS; sin él no se podría actualizar, reintentar ni eliminar un tenant sin duplicar recursos. Como contiene datos sensibles, se guarda fuera del repositorio y protegido de la siguiente forma:

Tabla 9. Estado de los motores de infraestructura

Aspecto	Implementación
Ubicación	Bucket S3 presencia-tfstate-aliaga23: tenants/<id>/ para Terraform y pulumi/ para Pulumi.
Cifrado	Cifrado del lado del servidor (AES-256) para todos los objetos.
Acceso	Acceso público bloqueado; solo el rol IAM de la plataforma y el administrador de la cuenta lo leen.
Historial	Versionado del bucket activado: cada cambio de estado conserva la versión anterior.
Bloqueo	Bloqueo nativo de S3 (use_lockfile) para que dos operaciones no modifiquen el mismo tenant.
Secretos en Pulumi	Cifrados con una frase de paso guardada en SSM Parameter Store.
Borrado accidental	El bucket tiene prevent_destroy en su definición de Terraform.

Tabla 10. Comandos principales

Acción	Comando
Construir la imagen	`packer build -var version_app=$(git describe) .`
Desplegar la plataforma	`terraform apply (environments/plataforma)`
Levantar en una PC	`docker compose up -d --build (plataforma)`
Bajar la plataforma	`terraform destroy (environments/plataforma)`

6. SEGURIDAD Y AISLAMIENTO

Tabla 11. Controles de seguridad

Control	Implementación
Aislamiento de red	Una VPC por tenant; sin conexión entre VPC.
Acceso a la base	Puerto 5432 solo desde el grupo de seguridad de la aplicación del mismo tenant.
Sesiones	Cada tenant firma sus sesiones con su propia clave JWT.
Secretos	Generados por el motor y guardados cifrados en SSM; nunca en el repositorio.
Permisos	Rol IAM por tenant que solo lee sus propios parámetros.
Escaneo previo	gitleaks detiene el despliegue si encuentra secretos; checkov advierte.
Repositorio	gitleaks revisa todo el historial en cada push y pull request.
Servidores	IMDSv2 obligatorio, discos cifrados, firewall ufw y SSH solo desde la plataforma.
Errores sin secretos	Los motores ocultan los valores sensibles; gitleaks enmascara lo que detecta; los mensajes al usuario no incluyen credenciales.

El aislamiento se verifica con un script automático que intenta, desde un tenant, entrar al otro con sus credenciales y con su token de sesión, alcanzar la base de datos y la API interna desde internet y conectarse desde su servidor a los puertos del otro tenant. En la ejecución registrada se aprobaron las 20 comprobaciones.

7. MONITOREO Y ALERTAS

Prometheus descubre los servidores encendidos por sus etiquetas de AWS, de modo que los tenants nuevos aparecen solos y los suspendidos no generan falsas alarmas. Grafana muestra el tablero «Salud de los tenants», con filtros por entorno y por tenant: disponibilidad de cada servidor, estado de la base de datos, CPU, memoria, disco, peticiones, errores del backend y latencia. Está publicado en `http://3.235.12.162/grafana/d/salud-tenants`.

Tabla 12. Reglas de alerta

Alerta	Condición
Tenant caído	Un servidor no responde durante 2 minutos.
CPU alta	Uso de procesador mayor a 85 % durante 5 minutos.
Disco casi lleno	Uso de disco mayor a 85 % durante 5 minutos.
Errores 5xx	Más del 5 % de las peticiones terminan en error durante 5 minutos.
Base de datos caída	La API de un tenant no llega a su base durante 2 minutos.
Memoria alta	Uso de memoria mayor a 90 % durante 5 minutos.

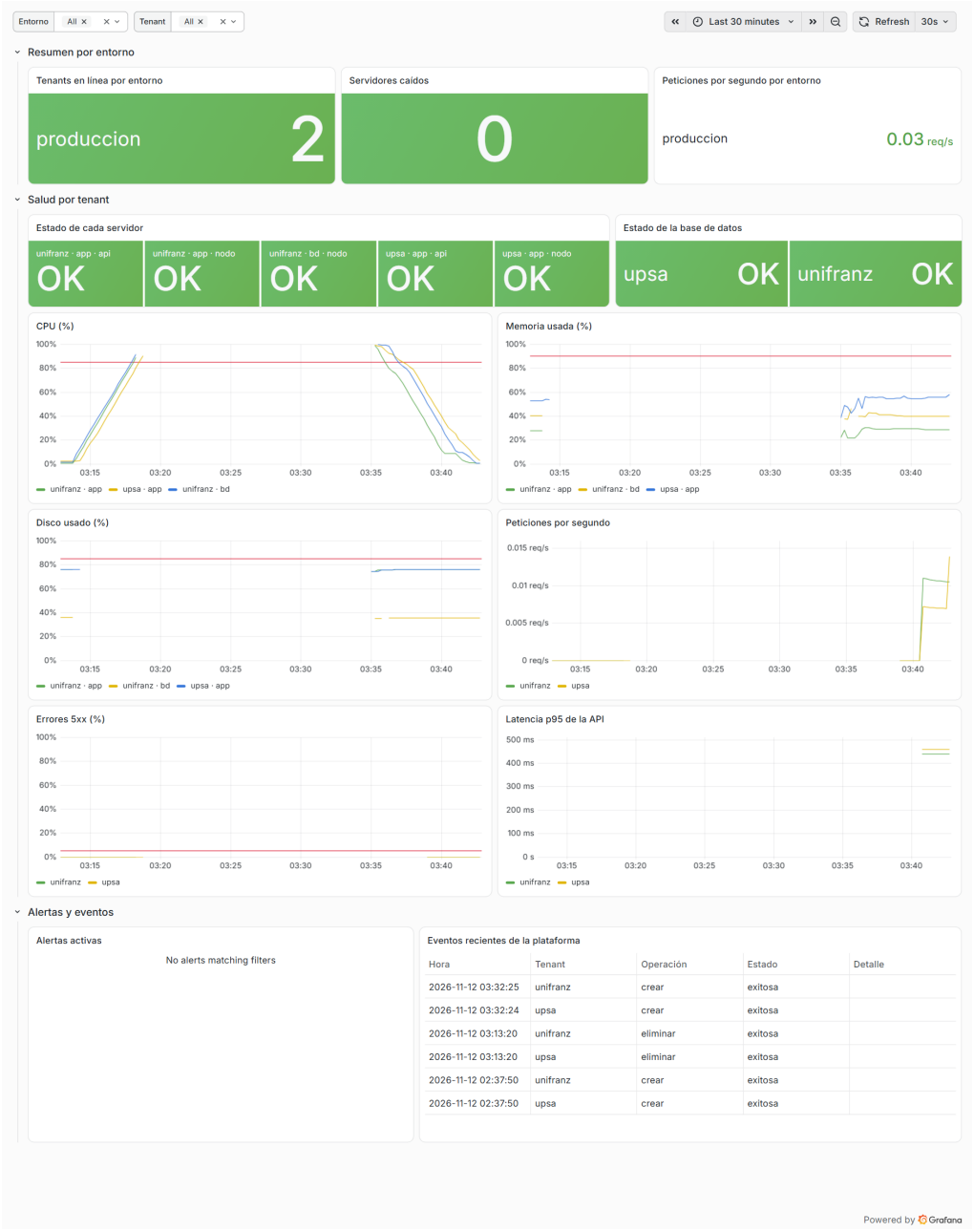


Figura 4. Tablero de salud de los tenants en Grafana.

8. MANUAL DE USUARIO

8.1. Página de inicio y catálogo

La dirección `http://3.235.12.162` muestra la página de inicio con el catálogo de planes, sus recursos y su costo. El botón **Ingresar a la consola** lleva al inicio de sesión.

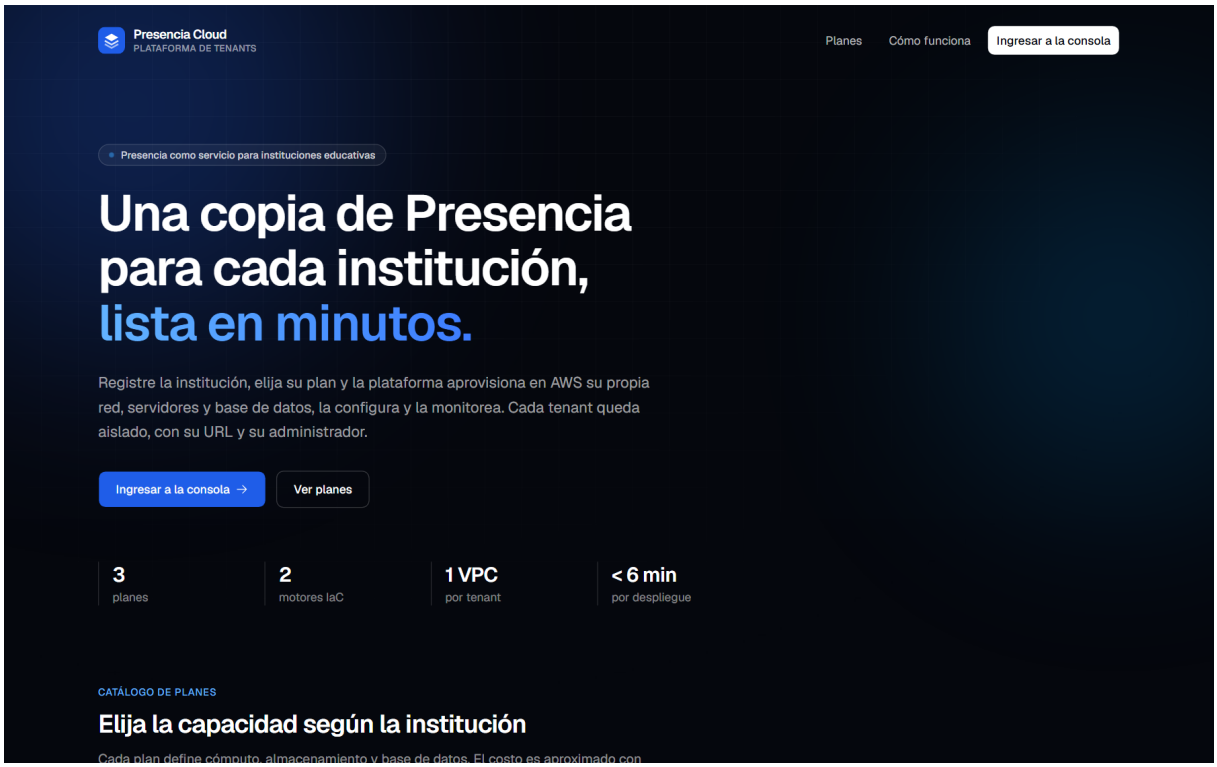


Figura 5. Página de inicio de la plataforma.

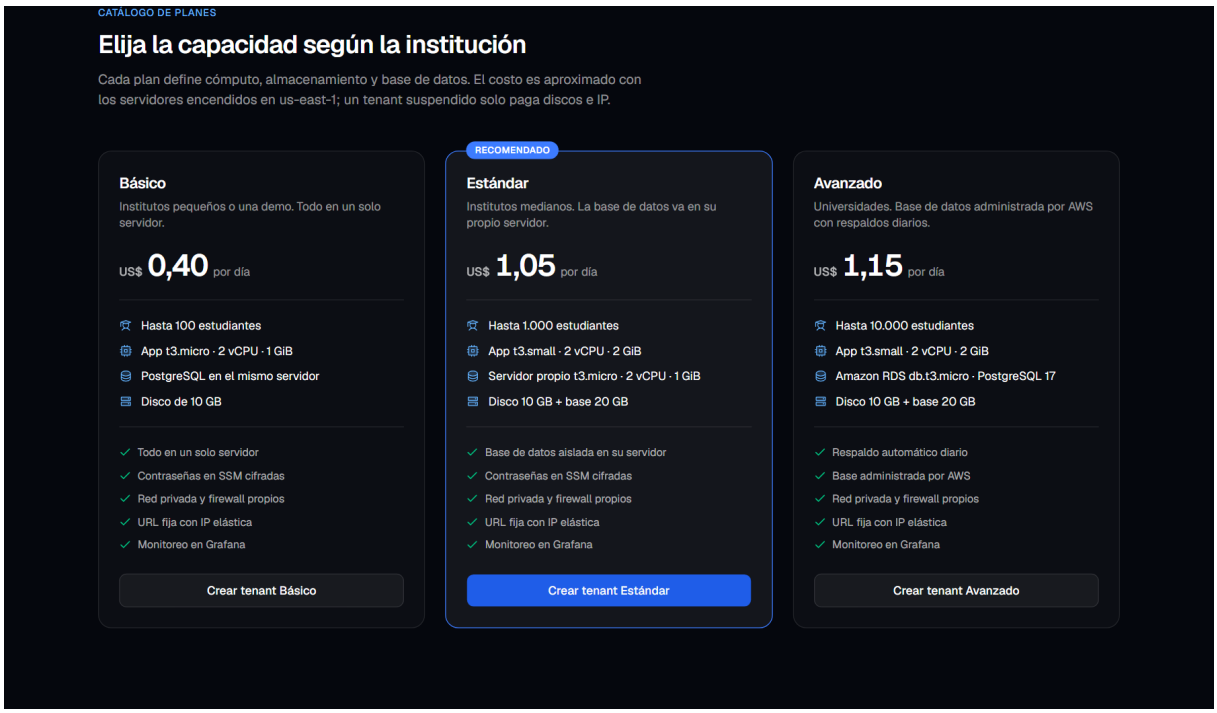


Figura 6. Catálogo de planes en la página de inicio.

8.2. Ingreso

Se ingresa con el usuario **admin** y la contraseña **admin1234!**. La sesión dura 12 horas y se cierra desde el menú del usuario, abajo a la izquierda.

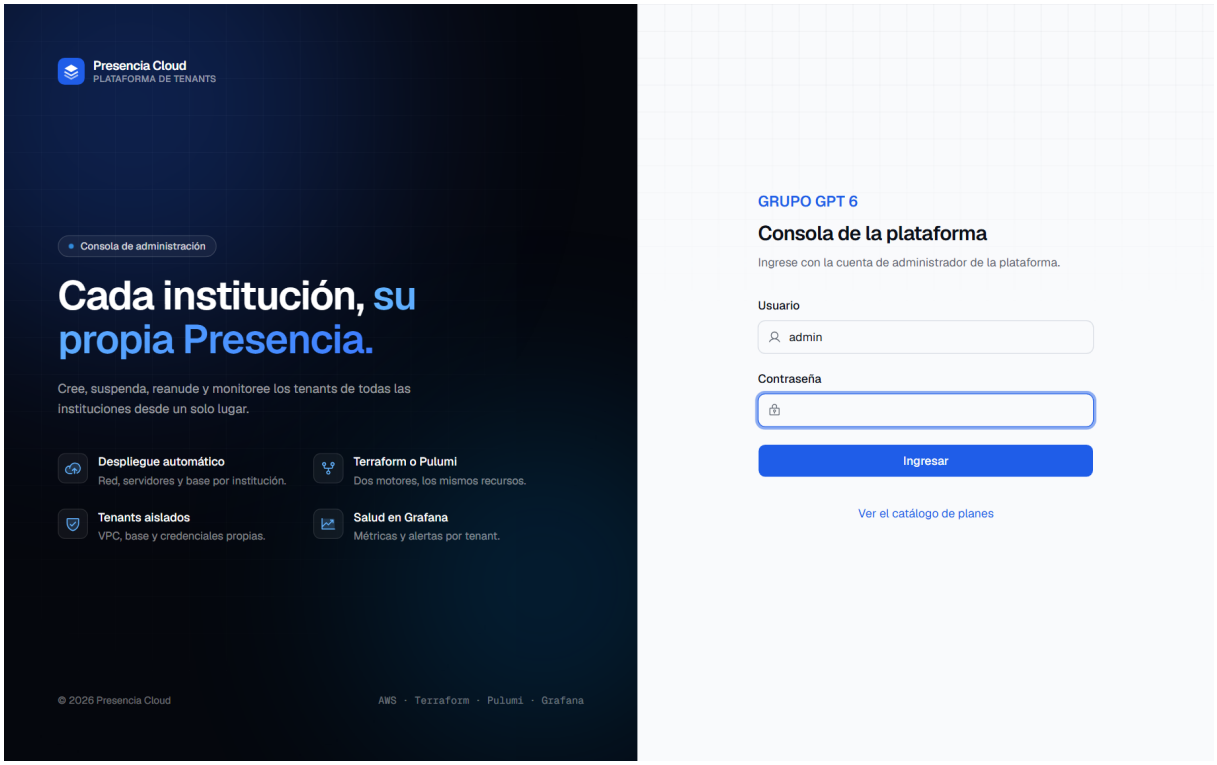


Figura 7. Inicio de sesión de la consola.

8.3. Panel

El panel resume los tenants activos, suspendidos y con error, el costo estimado por día, las instituciones, las alertas activas y la actividad reciente. El menú lateral da acceso a todas las secciones y puede contraerse.

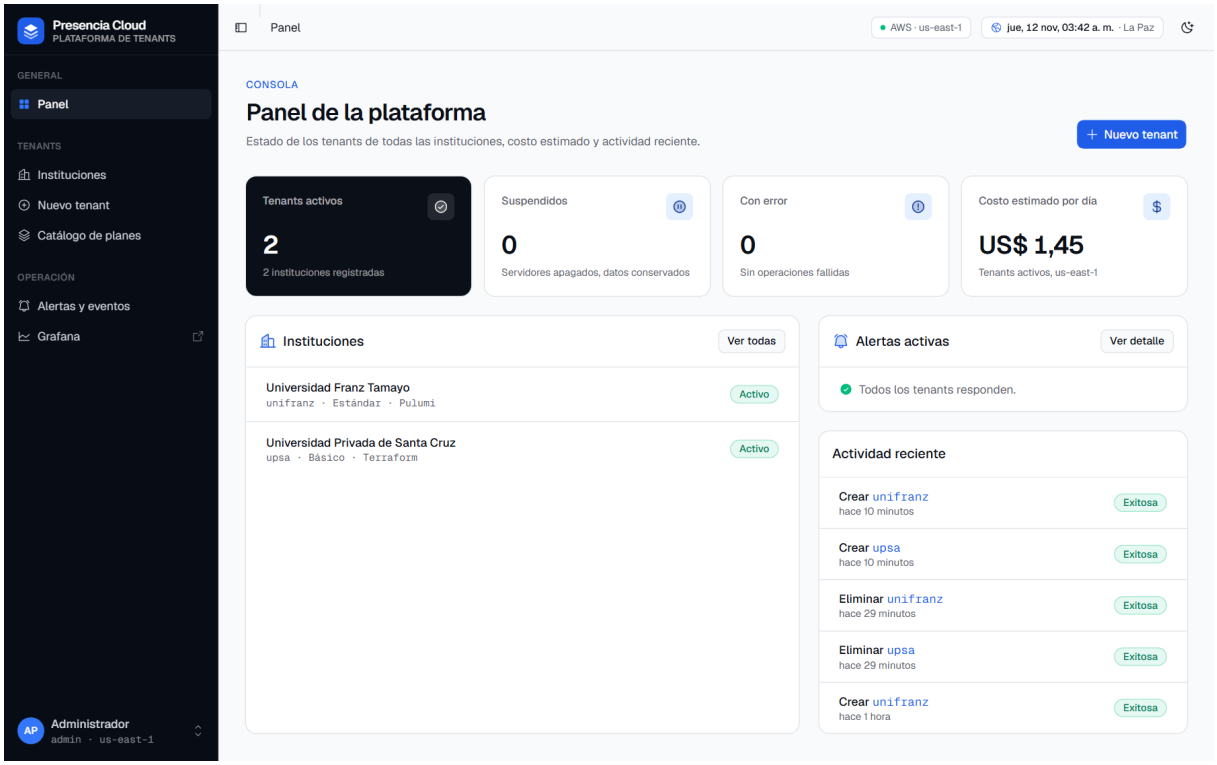


Figura 8. Panel de la plataforma.

8.4. Crear un tenant

1. Abrir **Nuevo tenant** y hacer clic en la tarjeta del plan.
2. Completar el nombre de la institución; el identificador se sugiere solo y no se puede cambiar después. Indicar el correo, el registro y el nombre del administrador del tenant.
3. Elegir región, entorno y motor (Terraform o Pulumi) y, si se desea, los datos de ejemplo.
4. Revisar el resumen con el costo y presionar **Crear tenant**.

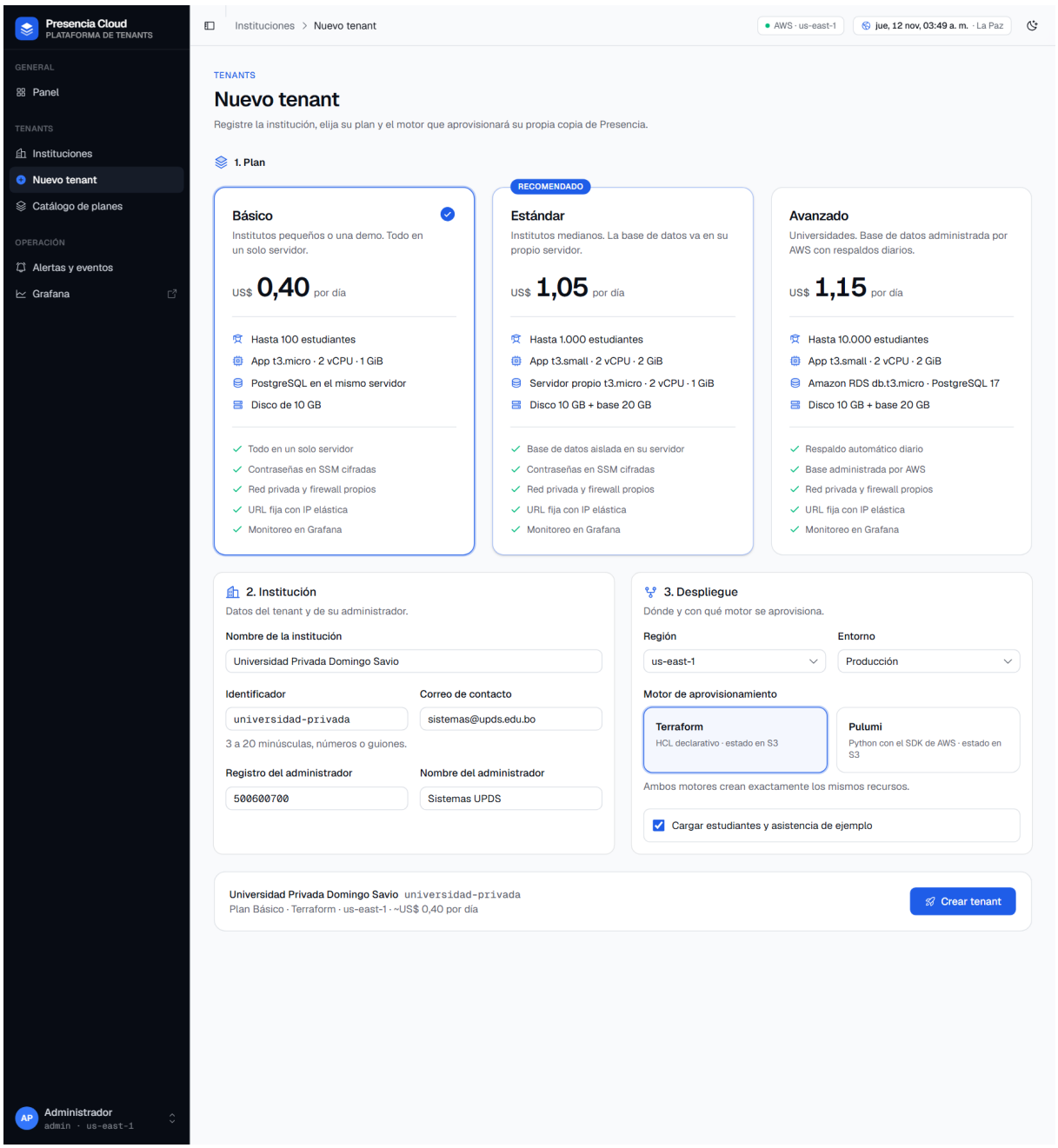


Figura 9. Formulario de un nuevo tenant.

8.5. Seguimiento del aprovisionamiento

La creación se ejecuta en segundo plano. La pantalla del tenant muestra el log en vivo con el escaneo de seguridad, cada recurso creado y la espera hasta que la aplicación responde. Un tenant Básico o Estándar tarda entre 4 y 6 minutos; uno Avanzado, entre 15 y 20 por la base RDS.

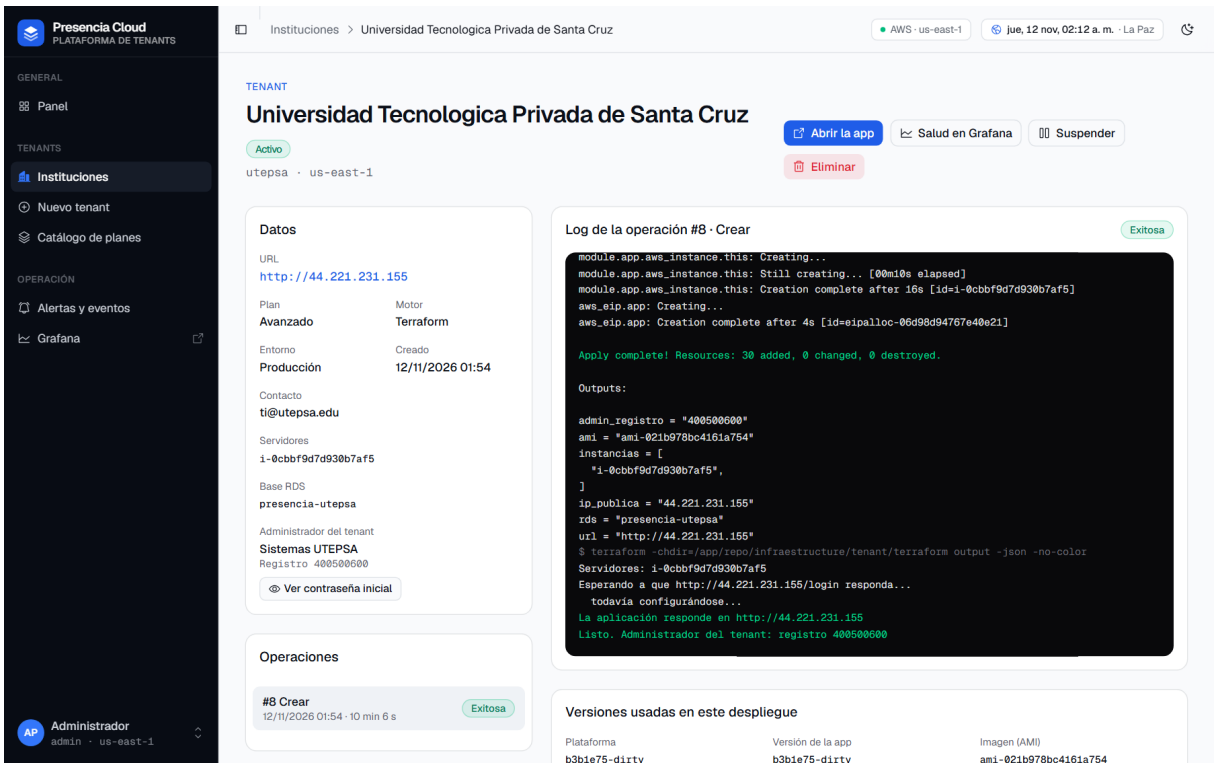


Figura 10. Registro de creación de un tenant con base RDS.

Tabla 13. Estados de un tenant

Estado	Significado
Creando	Se están creando sus recursos.
Activo	Funciona y su dirección se puede usar.
Suspendiendo / Suspendido	Se apagan / están apagados sus servidores; los datos se conservan.
Reanudando	Se encienden de nuevo.
Eliminando / Eliminado	Se borra / se borró su infraestructura.
Error	La última operación falló; se puede reintentar.

8.6. Uso del tenant

Cuando el estado es **Activo** se entrega a la institución su dirección, el registro de su administrador y la contraseña común **admin1234!**, que también reciben los estudiantes importados. La pantalla registra además las versiones usadas en el despliegue.

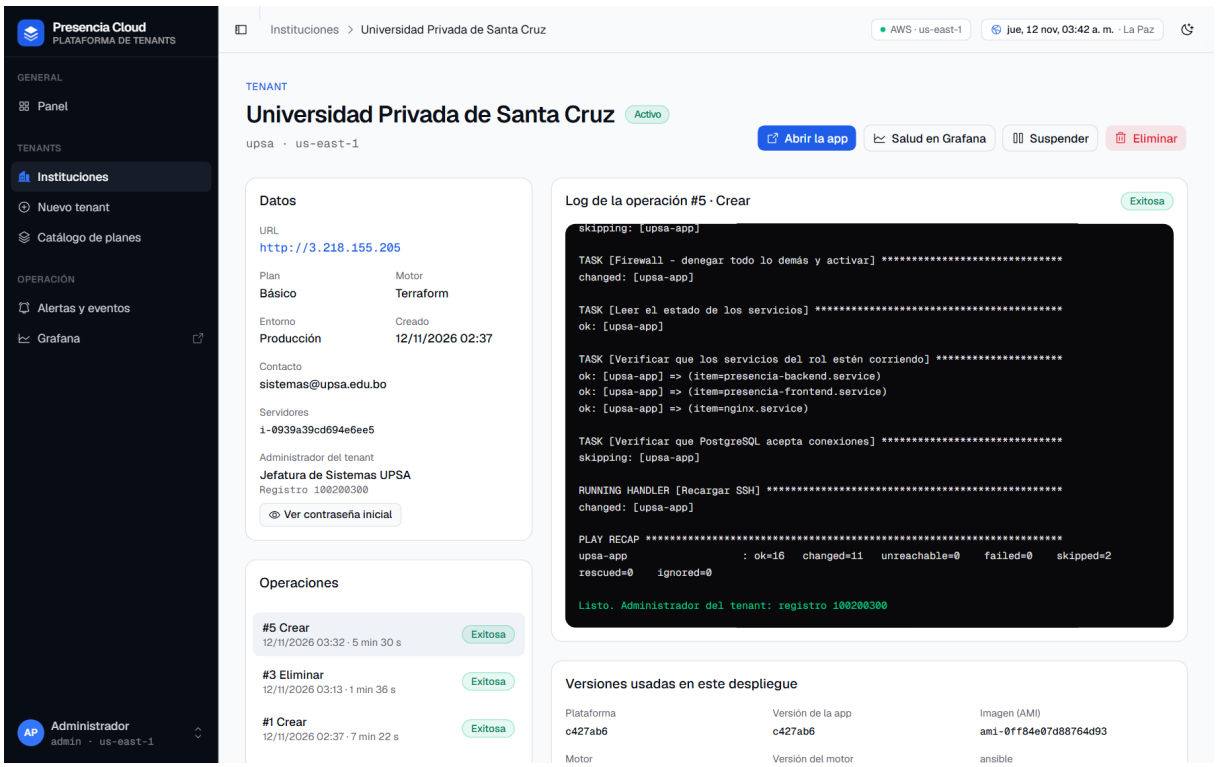


Figura 11. Detalle de un tenant activo.

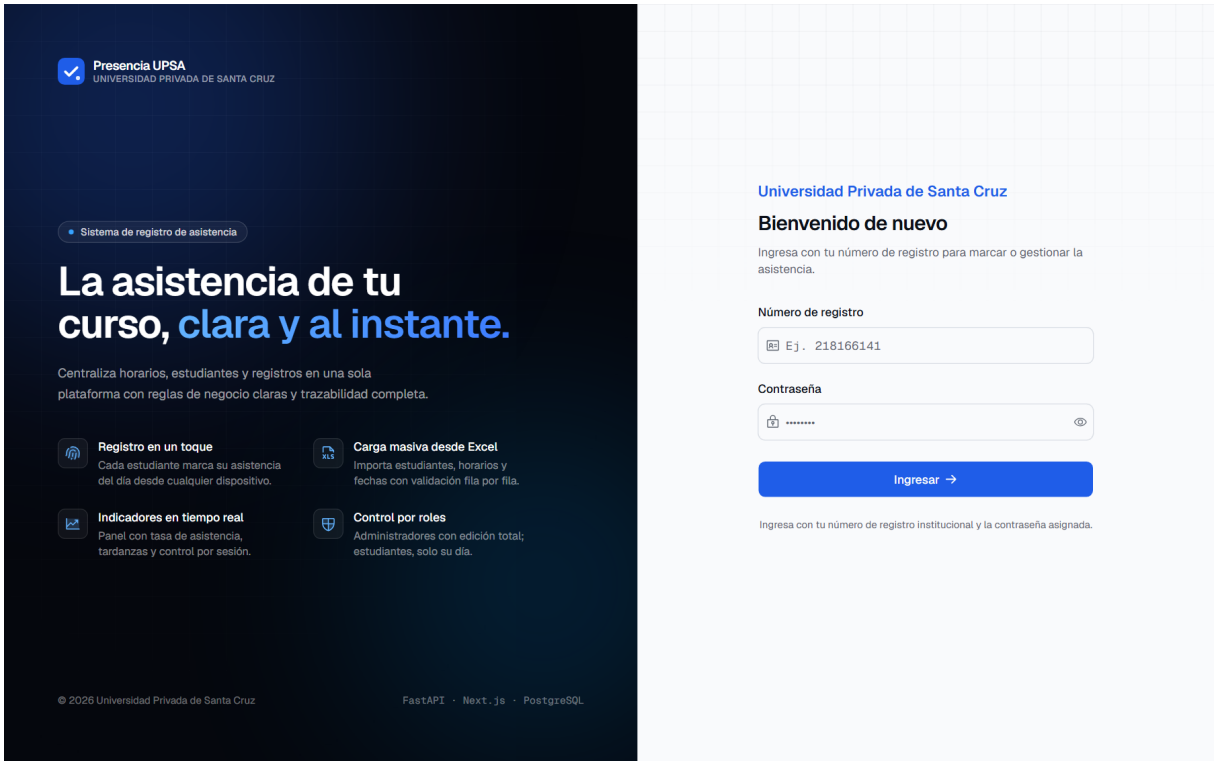


Figura 12. Inicio de sesión del tenant con la marca de su institución.

Dentro de su Presencia, el administrador del tenant ve en el panel el **estado de sus servicios**: si la API y la base de datos responden, la latencia de la base y la versión instalada.

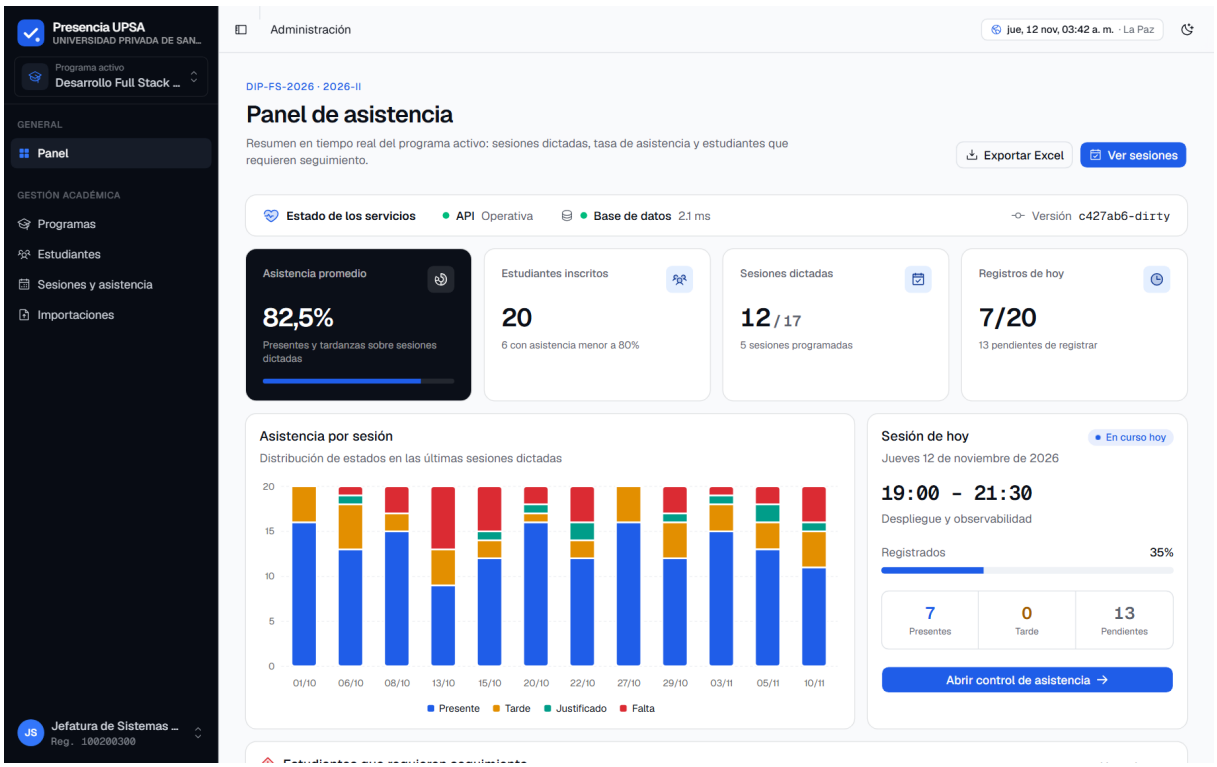


Figura 13. Panel del administrador del tenant con el estado de sus servicios.

8.7. Búsqueda y edición

Instituciones lista todos los tenants con búsqueda por nombre o identificador y filtros por estado, plan y entorno. El lápiz de cada fila permite corregir el nombre de la institución.

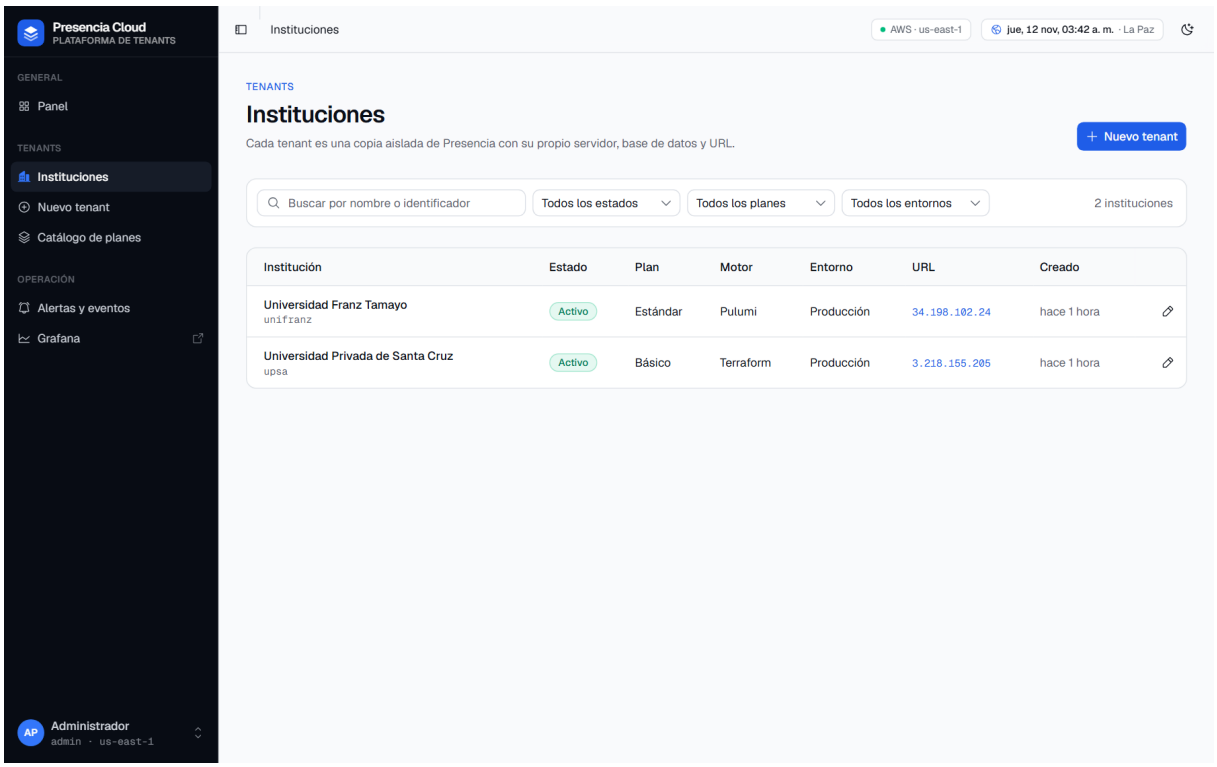


Figura 14. Lista de instituciones.

8.8. Suspender, reanudar y eliminar

Suspender apaga los servidores para no pagar por ellos; **Reanudar** los enciende en la misma dirección y con todos los datos. **Eliminar** muestra la lista de recursos que se borrarán y exige escribir el identificador; la acción no se puede deshacer.

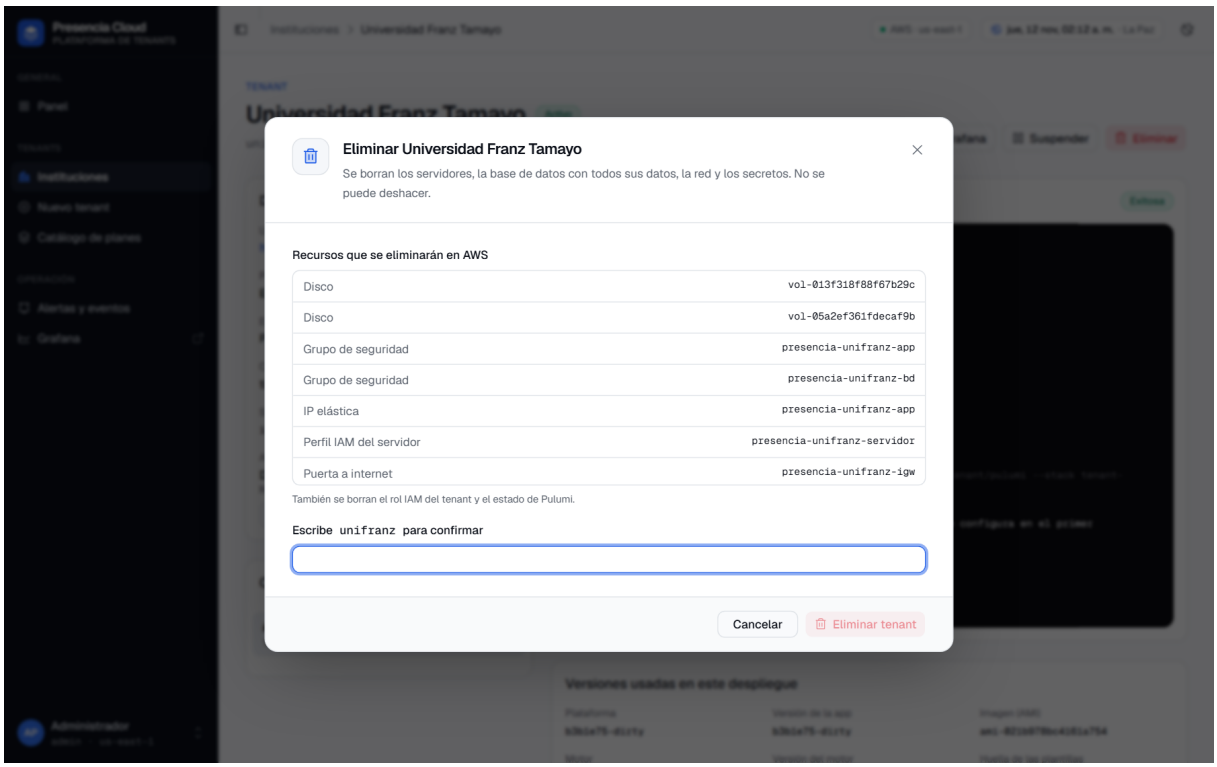


Figura 15. Confirmación de eliminación con los recursos afectados.

8.9. Reintentar una operación fallida

Si una operación falla, el tenant queda en estado **Error** y se muestra la causa en lenguaje sencillo. Tras corregirla, **Reintentar** continúa desde el estado guardado sin duplicar lo ya creado.

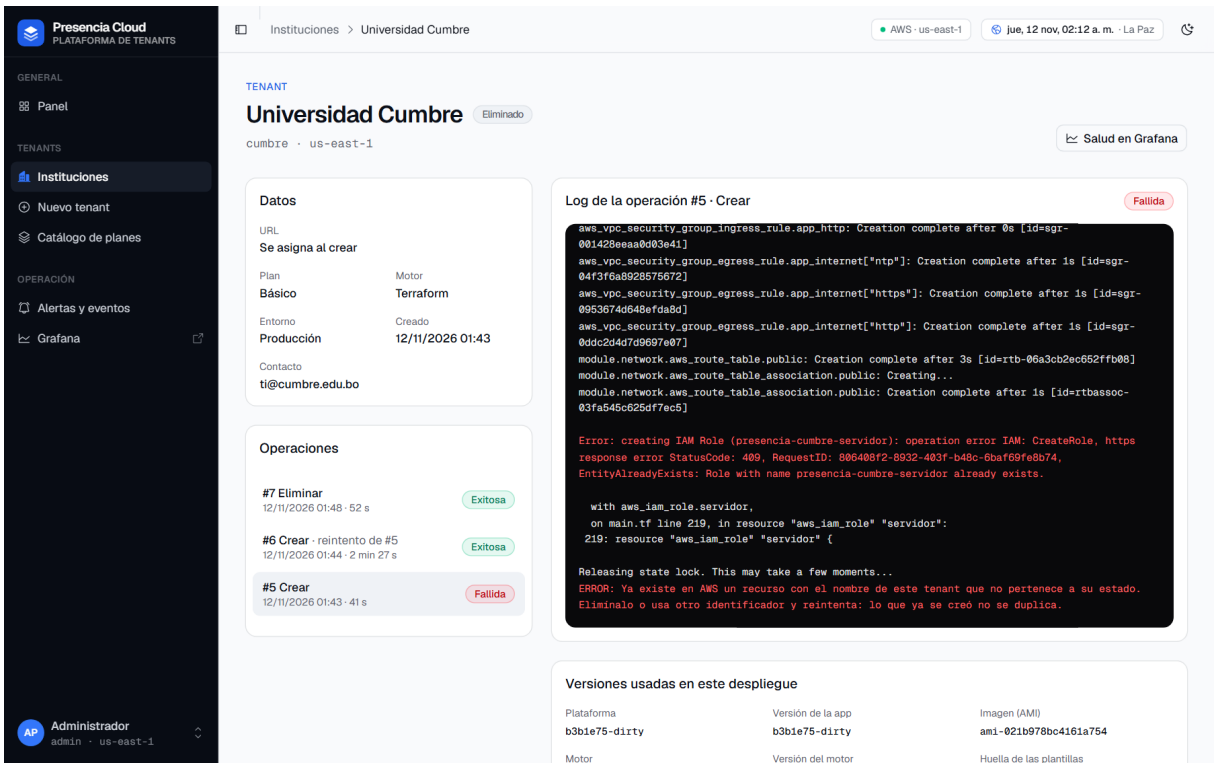


Figura 16. Operación fallida con su explicación y su reintento.

Tabla 14. Mensajes de error frecuentes

Mensaje	Acción
Credenciales de AWS no válidas	Revisar el rol o las credenciales de la plataforma.
Ya existe un recurso con ese nombre	Eliminar el recurso ajeno o usar otro identificador.
Límite de VPC, IP elásticas o vCPU	Eliminar o suspender tenants que no se usen.
AWS sin capacidad o tardó demasiado	Esperar unos minutos y reintentar.
Secretos escritos en la configuración	Quitarlos y guardarlos en SSM.

8.10. Alertas y eventos

La sección **Alertas y eventos** reúne las alertas activas de Grafana, con el tenant y la hora, y el historial de operaciones de todas las instituciones.

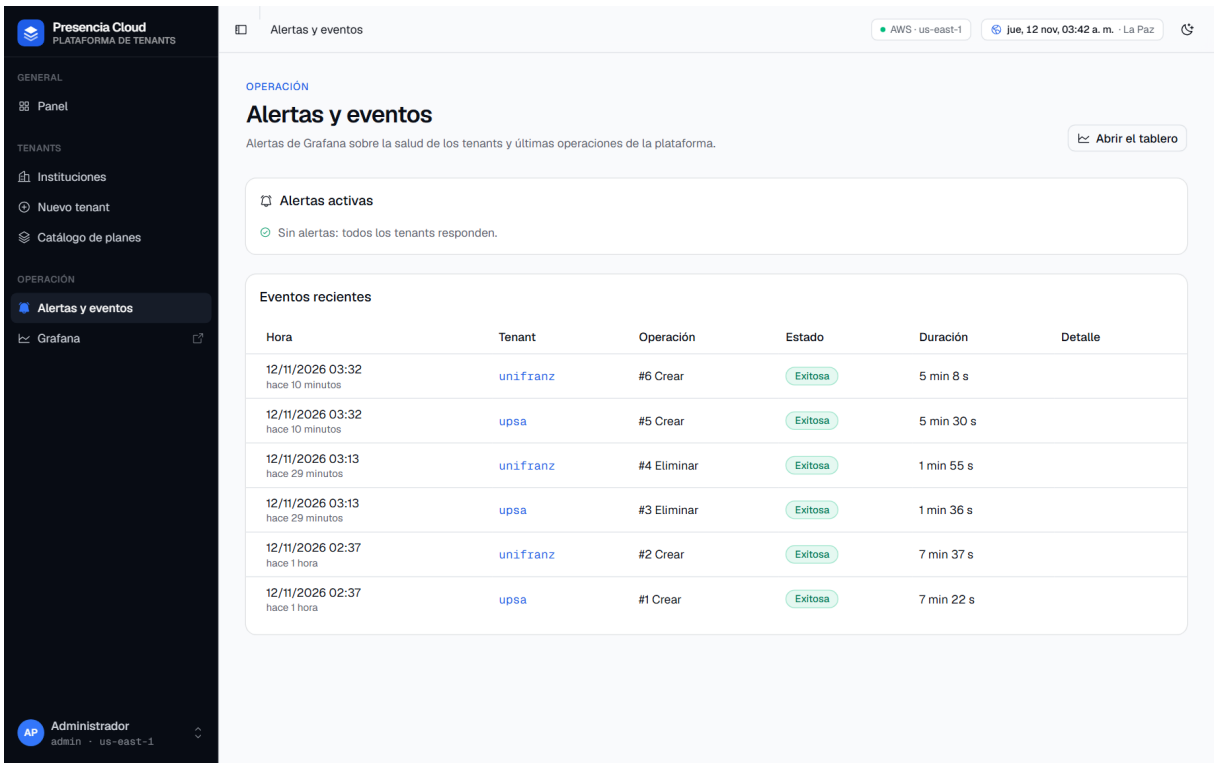


Figura 17. Alertas y eventos recientes.

9. PRUEBAS Y RESULTADOS

Las pruebas se ejecutaron en la cuenta de AWS del proyecto, región us-east-1.

Tabla 15. Criterios de aceptación y resultados

Criterio	Prueba	Resultado
Crear tenants con distintos planes	Básico con Terraform, Estándar con Pulumi y Avanzado con RDS	Aprobado
Desplegar frontend, backend y base	Ingreso del administrador y lista de estudiantes por tenant	Aprobado
Suspender y reanudar	Servidores detenidos; vuelve en la misma URL con sus datos	Aprobado
Salud en Grafana por tenant	Métricas de CPU, memoria, disco, peticiones y latencia	Aprobado
Alertas	Backend detenido: alerta «Tenant caído» con tenant y hora	Aprobado
Aislamiento entre tenants	Script de 20 comprobaciones, incluido SSH entre tenants	20 de 20
Reintento sin duplicar	Falla provocada y reintento: solo 6 recursos faltantes, misma VPC	Aprobado
Eliminar tenant	23 a 30 recursos destruidos y estado borrado	Aprobado
Escaneo de secretos	gitleaks sobre plantillas, configuración e historial	Sin hallazgos
Configuración con Ansible	Playbook posterior sobre cada servidor del tenant	failed=0 en todos
Estado de la base de datos	Panel y alerta en Grafana; tarjeta en el panel del tenant	Aprobado

10. CONCLUSIONES

- La plataforma automatiza de punta a punta la entrega de Presencia a nuevas instituciones: una institución queda operativa en minutos y con su propia marca.
- Definir el catálogo y la infraestructura como código permite que dos motores distintos, Terraform y Pulumi, produzcan exactamente la misma infraestructura.
- La imagen dorada separa la instalación, que se hace una vez, de la configuración de cada tenant, lo que reduce el tiempo de despliegue.
- El aislamiento por VPC, credenciales y claves propias, junto con el escaneo previo, protege los datos de cada institución.
- El monitoreo con descubrimiento automático y las alertas permiten operar varios tenants sin revisarlos uno por uno.